PROJECT REPORT

Fake Review Detector

An explainable web application that flags fake and paid product reviews from pasted text or screenshots, using machine learning and rule-based checks.

Date: October 2026

Technology: Python, Streamlit, scikit-learn, RapidOCR, VADER, Plotly

Deployment: Streamlit Community Cloud (free tier)

Contents

1. Abstract 3
2. Introduction 4
2.1 Problem statement 4
2.2 Objectives 4
2.3 Scope 4
3. Requirements 5
3.1 Functional requirements 5
3.2 Non-functional requirements 5
4. Technology Stack 6
5. System Architecture 7
6. Methodology 8
6.1 Dataset 8
6.2 Machine-learning model 8
6.3 Rule-based criteria 8
6.4 Combining the scores 9
6.5 Reading screenshots (OCR) 9
7. User Interface 11
8. Results and Evaluation 14
8.1 Machine-learning model 14
8.2 Full detector by strictness 14
8.3 Example cases 14
9. Testing 16
10. Deployment 17
11. Limitations and Ethical Considerations 18
12. Future Enhancements 19
13. Conclusion 20
References 20
Appendix A. Project Structure 21

1. Abstract

Online shoppers rely heavily on product reviews, but a growing share of those reviews are fake: written by bots, paid reviewers or sellers promoting their own products. This project builds the Fake Review Detector, a web application where a user pastes a review or uploads a screenshot of one and receives a FAKE or REAL verdict with a clear explanation of why.

The detector combines two complementary approaches. A machine-learning model (TF-IDF features with logistic regression) trained on 40,432 labelled reviews identifies machine-generated text with 95.1% accuracy and a ROC AUC of 0.989 on held-out data. Eleven explainable rule-based checks catch human-written fakes: exaggerated language, template phrases, missing details, one-sided sentiment, spam links, paid-review disclosures and star ratings that contradict the text. Screenshots are read with an on-device OCR engine. The interface, built in Streamlit, highlights the suspicious phrases, charts every criterion and lets the user adjust how strict the detector is.

2. Introduction

2.1 Problem statement

Reviews strongly influence purchase decisions, which makes them a target for manipulation. Fake reviews take several forms:

- **Machine-generated reviews:** text produced by language models at scale, often fluent but generic.
- **Paid or incentivized reviews:** written by people rewarded with money, free products or discounts.
- **Spam reviews:** reviews used as advertising, containing links, promo codes or contact details.
- **Sabotage reviews:** negative reviews posted by competitors, often with a rating that does not match the text.

A shopper has no easy way to tell these apart from genuine experiences. The goal of this project is to give them a simple tool that does, and that explains its reasoning instead of acting as a black box.

2.2 Objectives

- Accept a review as pasted text or as a screenshot image.
- Classify the review as FAKE or REAL with a confidence level.
- Explain the verdict using understandable criteria and highlight the phrases responsible.
- Keep the technology stack simple (Python only) and deployable for free.
- Present the results in a colorful, interactive interface suitable for a project showcase.

2.3 Scope

The application targets English-language product reviews, such as those on e-commerce sites. It analyzes one review at a time and does not scrape websites, look up reviewer histories or access any external service, so every result can be reproduced from the review text alone.

3. Requirements

3.1 Functional requirements

ID	Requirement
F1	The user can paste review text into a text box and request an analysis.
F2	The user can upload a screenshot (PNG, JPG or WEBP up to 10 MB); the app extracts the review text with OCR and lets the user correct it before analysis.
F3	The user can optionally supply the star rating; it is read automatically from screenshots that show "x out of 5".
F4	The app shows a FAKE / REAL verdict, a fake score from 0% to 100% and a confidence level.
F5	The app explains the verdict: per-criterion scores, the reasons behind them, and highlighted phrases.
F6	The user can choose how strict the detector is (Relaxed, Balanced, Strict).
F7	One-click examples demonstrate suspicious, genuine and spam reviews.
F8	A session history lists every review checked during the visit.

3.2 Non-functional requirements

Quality	Target
Explainability	Every verdict lists the criteria that drove it, in plain English.
Simplicity	Pure Python stack; no database, no API keys, no paid services.
Portability	Runs on Streamlit Community Cloud with no system packages; OCR models ship inside the Python package.
Privacy	Reviews are processed in memory and never stored or sent to third parties.
Usability	Works on desktop and mobile, and in light and dark themes.
Reliability	Automated tests cover the detector, the OCR pipeline and the full app.

4. Technology Stack

All components are open-source Python libraries, which keeps the project free to build, run and host.

Layer	Technology	Purpose
Language	Python 3.11	Single language for model, logic and interface
Web interface	Streamlit 1.66	Turns the Python script into an interactive web page
Charts	Plotly 7.1	Interactive criteria and sentence-tone charts with hover details
Machine learning	scikit-learn 1.9	TF-IDF vectorizers and logistic regression classifier
Sentiment	VADER 3.3	Sentence-level positive / negative tone scoring
OCR	RapidOCR 3.10 + ONNX Runtime	Reads text from screenshots locally, with no external API
Data handling	pandas, NumPy, Pillow	Dataset loading, image processing, history table
Testing	pytest + Streamlit AppTest	Unit tests and a headless end-to-end app test
Hosting	Streamlit Community Cloud	Free public hosting directly from a GitHub repository

5. System Architecture

The application is organized as a small Python package with a clear separation between the interface, the analysis logic and the two detectors.

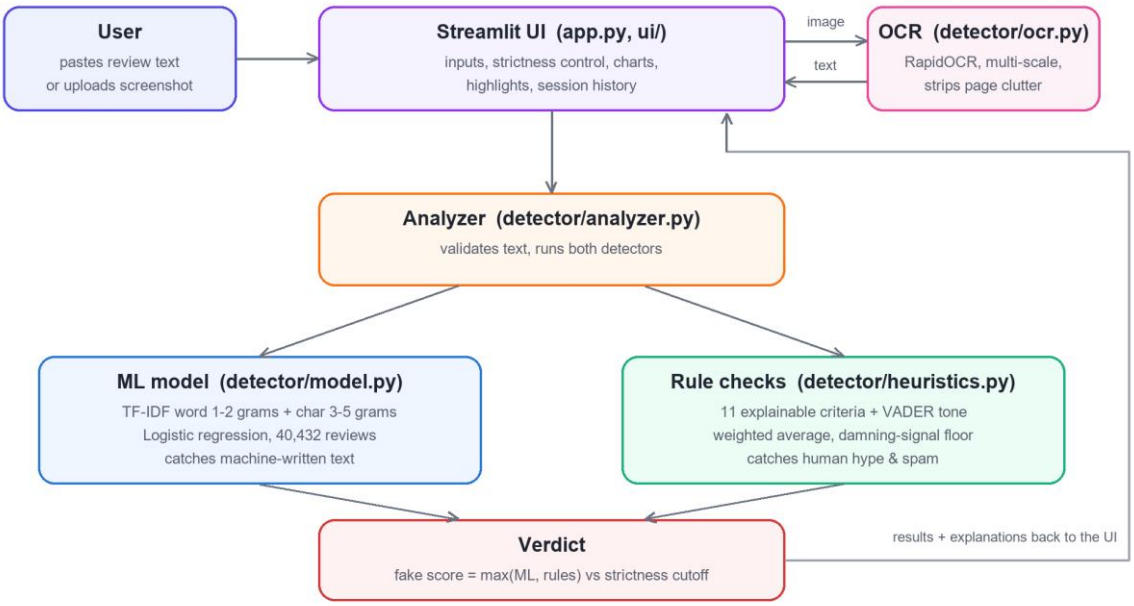


Figure 1. System architecture and data flow

Module	Responsibility
app.py	Streamlit page: input tabs, strictness control, result layout, session history
ui/components.py	Verdict card, stat tiles, highlighted review, Plotly charts
ui/style.css	Gradient header, cards, animations and light/dark colors
detector/analyzer.py	Validates input, runs both detectors, combines them into a verdict
detector/model.py	Loads the trained classifier and returns the machine-generated probability
detector/heuristics.py	The eleven rule-based criteria and how they are combined
detector/ocr.py	Screenshot to text: multi-scale OCR, line grouping, clutter removal
train_model.py	Downloads the dataset, trains, evaluates and saves the model
tests/	Unit tests for the detector and OCR, and an end-to-end app test

A request flows as follows: the user submits text (or an image, which the OCR module converts to text); the analyzer runs the ML model and the rule checks on the text; the higher of the two scores becomes the fake score; the score is compared with the cutoff for the selected strictness level; and the interface renders the verdict together with the evidence.

6. Methodology

6.1 Dataset

The model is trained on the **Fake Reviews Dataset** published by Salminen et al. (2022). It contains 40,432 Amazon product reviews across 10 product categories, split evenly between 20,216 original reviews written by real customers and 20,216 reviews generated by a language model. Each record contains the category, the star rating, the label and the review text.

Because the dataset's "fake" reviews are computer-generated, the trained model specializes in spotting machine-written text. Human-written fakes are handled by the rule-based checks described in section 6.3.

6.2 Machine-learning model

- **Word features:** TF-IDF over single words and word pairs (1-2 grams), up to 60,000 terms, with sub-linear term frequency.
- **Character features:** TF-IDF over 3-5 character sequences, up to 60,000 terms. These capture spelling, punctuation and style patterns typical of generated text.
- **Classifier:** logistic regression (C = 4), which is fast, works well on sparse text features and outputs a calibrated probability.
- **Training:** a stratified 80/20 split (32,345 training and 8,087 test reviews) for evaluation; the shipped model is then refit on all data. The saved model is 2.2 MB.

6.3 Rule-based criteria

Each rule examines one well-documented trait of deceptive reviews and returns a score from 0 (looks genuine) to 1 (looks fake) with a plain-English reason. Rules marked as evidence-only count only when they fire, because their absence says nothing about a review.

Criterion	What it detects	Weight	Type
Extreme / superlative words	Density of words such as amazing, perfect, worst	1.2	Scored
Stock / template phrases	Phrases such as "highly recommend", "worth every penny"	1.2	Scored
Excessive punctuation / CAPS	Exclamation marks per sentence and ALL-CAPS words	0.8	Scored
Self-focused language	Share of I / me / my (Ott et al., 2011)	0.5	Scored
Lack of concrete details	Few product features, numbers or durations of use	1.2	Scored
One-sided sentiment	Entirely positive or negative, with no trade-offs	1.0	Scored
Very short review	Too few words to show real use	0.7	Scored
Repetitive wording	Low vocabulary variety and repeated word pairs	0.8	Scored
Links / promo codes / contact info	URLs, emails, phone numbers, discount codes	1.5	Evidence-only

Criterion	What it detects	Weight	Type
Incentivized review disclosure	"Received free in exchange for an honest review"	1.0	Evidence-only
Star rating vs. text mismatch	A 5-star complaint or a 1-star rave	1.5	Evidence-only

Sentiment is measured with VADER sentence by sentence, because scoring a whole review at once lets a negation in one sentence ("never replied") flip the meaning of the next ("Terrible."). VADER's lexicon was also adjusted for product reviews; for example, "support" is treated as neutral and "broke" or "defective" as negative.

6.4 Combining the scores

The rule score is the weighted average of the criteria that carry evidence, with one safeguard: a damning evidence-only signal (such as a promo link or a rating mismatch) sets a minimum for the score, so it cannot be diluted by the checks that found nothing. Without this, an obvious spam review scored only 53%.

The final fake score is the higher of the ML probability and the rule score. The two detectors look for different kinds of fake, so a review is flagged when either one is confident. Averaging them would hide fakes that only one detector can see; for example, the model gives a human-written hype review a score of 0%.

The strictness setting decides the cutoff:

Strictness	Flags as fake when score is	Use when
Relaxed	65% or more	Only clear-cut cases should be flagged
Balanced (default)	50% or more	General use
Strict	35% or more	Catching more fakes matters more than false alarms

Confidence is measured as the distance between the score and the active cutoff, scaled from 50% (right at the cutoff) to 100% (at the end of the scale), so a verdict is never shown with less than 50% confidence.

6.5 Reading screenshots (OCR)

- **Engine:** RapidOCR, which runs PaddleOCR models on ONNX Runtime. The models are bundled in the Python package, so nothing is downloaded at runtime and no system installation is needed.
- **Multi-scale reading:** the recognizer occasionally garbles a line at one image size and reads it perfectly at another, and it reports low confidence when it fails. The image is therefore read at 1x, 1.5x and 2x, and the most confident reading is kept.

- **Layout:** detected text fragments on the same visual row are merged and ordered top to bottom.
- **Clean-up:** page clutter such as "Helpful", "Report" and "12 people found this helpful" is removed; the star rating and the "Verified Purchase" badge are extracted.
- **Human check:** the extracted text appears in an editable box, so the user can fix any OCR error before analysis.

7. User Interface

The interface is designed for a live demo: colorful, animated and readable in a few seconds, while every number remains explainable.

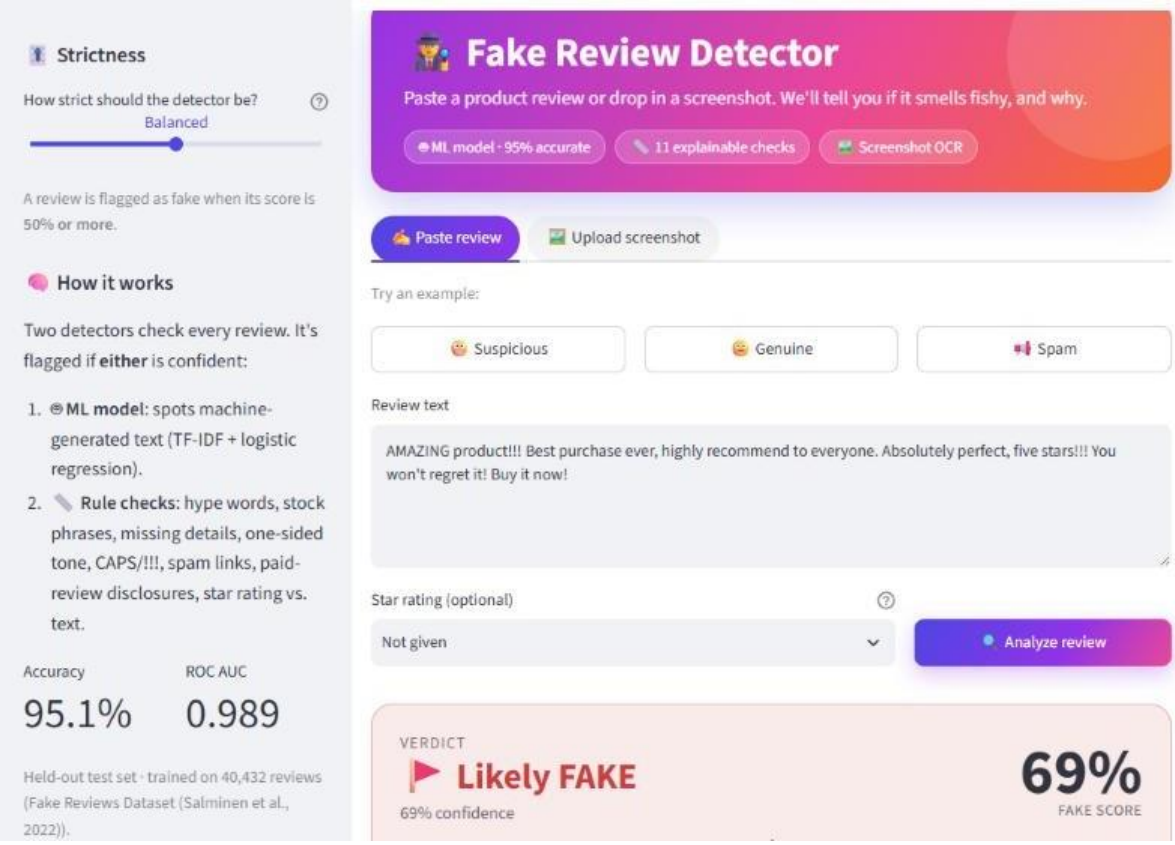


Figure 2. Home screen: gradient header, input tabs, examples and strictness control

After analysis, an animated verdict card shows the result, the fake score and a meter with the cutoff marked. Four tiles summarize the ML score, the rule score, the number of red flags and the overall tone. In the Highlights view, each suspicious phrase is colored by the check it triggered.

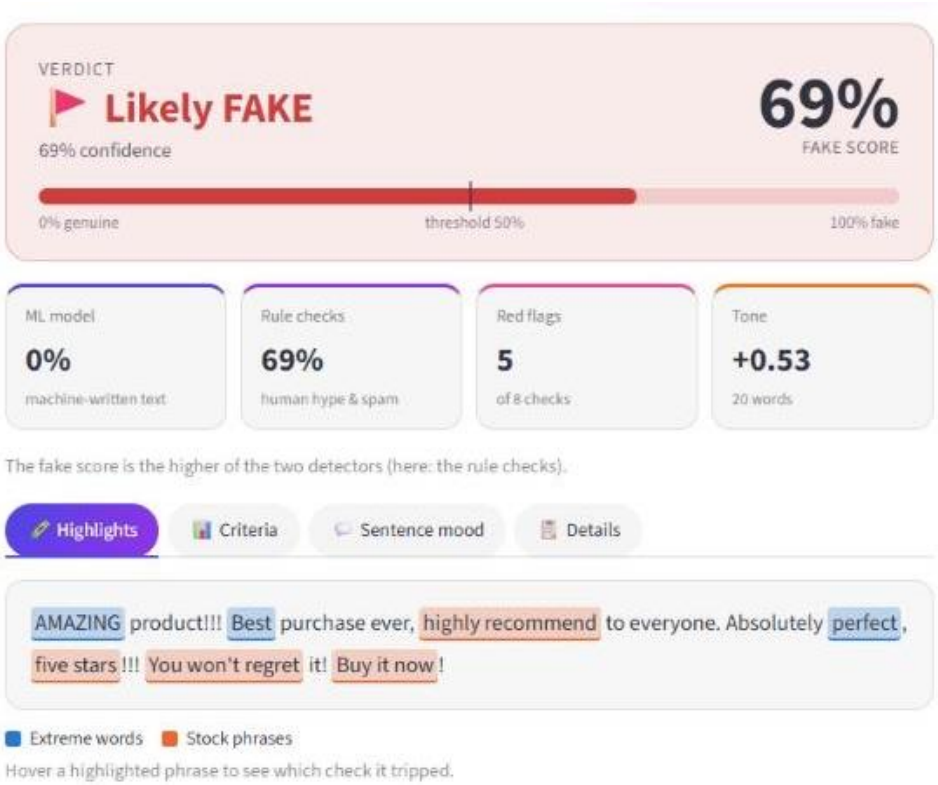


Figure 3. Verdict card, summary tiles and color-coded highlights

The Criteria view charts every check. Bars are colored by status (OK, Watch, Red flag) with matching icons, so the chart does not rely on color alone; hovering a bar shows the reasoning.



Figure 4. Interactive criteria chart

The Sentence mood view charts the tone of each sentence from negative to positive. The session history keeps a running table of every review checked.

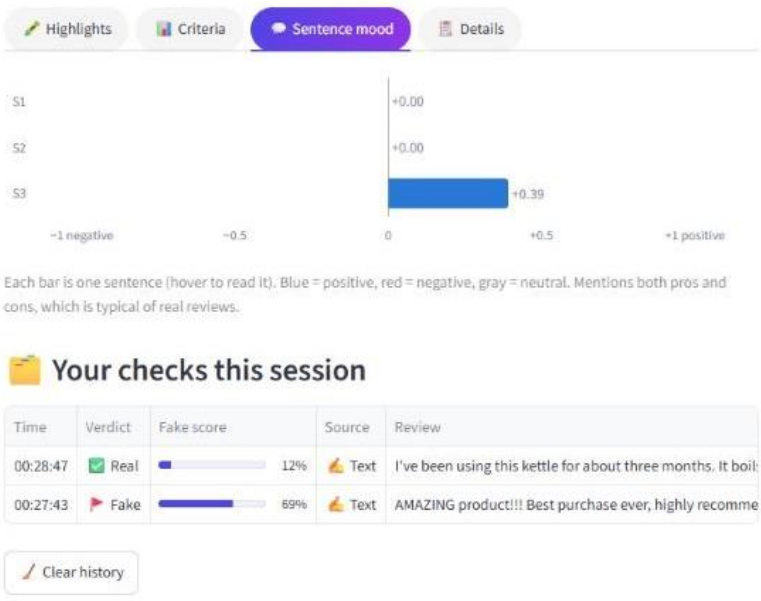


Figure 5. Sentence-tone chart and session history

In the screenshot tab, the uploaded image is read by OCR, the text is shown for correction, and the star rating is detected automatically.

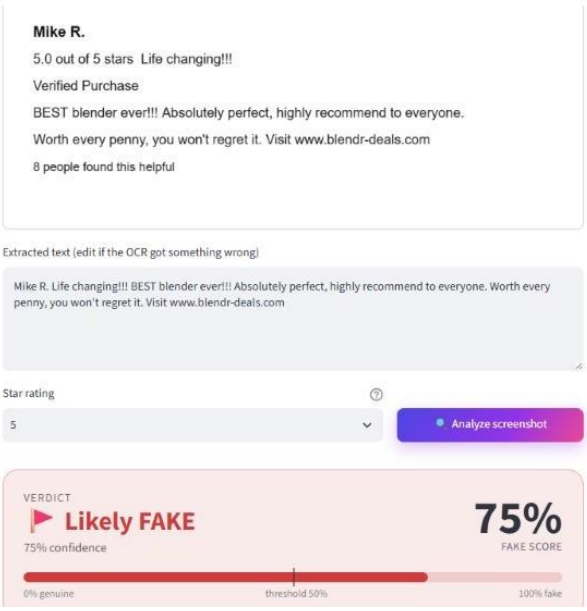


Figure 6. Screenshot analysis with OCR and automatic star-rating detection

Other interface details:

- Changing strictness updates the verdict instantly, without re-running the analysis.
- The layout adapts to phone screens and follows the viewer's light or dark theme.
- Highlight colors were checked with a color-vision-deficiency validator, and every color is paired with a label.
- All review text is HTML-escaped before display, so a malicious review cannot inject content into the page.

8. Results and Evaluation

8.1 Machine-learning model

Measured on the 8,087 held-out test reviews, which the model never saw during training:

Metric	Real reviews	Fake reviews	Overall
Precision	94.4%	95.8%	-
Recall	95.8%	94.3%	-
F1 score	0.951	0.950	0.950
Accuracy	-	-	95.1%
ROC AUC	-	-	0.989

Confusion matrix on the test set:

	Predicted real	Predicted fake
Actually real (4,044)	3,876 correct	168 false alarms
Actually fake (4,043)	230 missed	3,813 caught

8.2 Full detector by strictness

The complete detector (ML model plus rules) was run on a random sample of about 3,000 dataset reviews:

Strictness	Genuine reviews flagged as fake	Machine-written fakes caught
Relaxed	2.8%	93.4%
Balanced	4.7%	96.5%
Strict	12.2%	98.4%

Inspection of the genuine reviews flagged at Balanced showed that many openly disclose being incentivized (for example, "received this item at a discount in exchange for an honest review"). These are human-written, but they are not independent, so flagging them is intended. One false alarm, a book's ISBN being matched as a phone number, led to a stricter phone-number pattern.

8.3 Example cases

Results at the Balanced setting:

Review	ML	Rules	Final	Verdict	Main reasons
Hype review	0%	69%	69%	FAKE	Extreme words, stock phrases
Spam with promo code	29%	95%	95%	FAKE	Link / promo code, no concrete details
Incentivized review	87%	65%	87%	FAKE	Paid-review disclosure, one-sided tone

Review	ML	Rules	Final	Verdict	Main reasons
5-star complaint	1%	90%	90%	FAKE	Rating contradicts text, extreme words
Detailed genuine review	12%	1%	12%	REAL	Specific details, pros and cons
Genuine complaint (screenshot)	2%	7%	7%	REAL	Specific details, consistent 1-star rating

The hype review shows why two detectors are needed: the ML model scores it 0% because it is human-written, but the rules catch it. The incentivized review shows the reverse: the ML model is the stronger signal.

9. Testing

The project has 15 automated tests (pytest), all passing:

- **Detector tests:** hype and spam reviews are flagged; a detailed, balanced review is not; rating mismatches fire; too-short input is rejected.
- **Regression tests:** spam is not averaged away even on Relaxed; an ISBN is not treated as a phone number; confidence never drops below 50% at any strictness level.
- **OCR test:** a sample screenshot is read correctly, clutter is removed and the 5-star rating and Verified Purchase badge are detected.
- **End-to-end test:** Streamlit's AppTest runs the whole page headless, clicks an example and checks that a FAKE verdict and a history row appear.

In addition, the interface was checked manually in a browser in light mode, dark mode and at phone width, and every pinned dependency was confirmed to have Linux builds for Python 3.11 and 3.12, the versions available on Streamlit Community Cloud.

10. Deployment

The application is designed for Streamlit Community Cloud, which hosts public Streamlit apps for free directly from GitHub. No secrets, API keys or system packages are required.

1. Push the project folder to a public GitHub repository, including models/review_model.joblib (the dataset folder is excluded by .gitignore).
2. Sign in at share.streamlit.io with the GitHub account and choose Create app.
3. Select the repository and branch, and set the main file path to app.py.
4. Under Advanced settings, choose Python 3.11 (the version used to train the model).
5. Click Deploy. Streamlit installs requirements.txt and publishes the app at a public streamlit.app address.

To run the project locally:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

scikit-learn is pinned to the version that trained the model, because a saved model only loads reliably with that version. Upgrading it requires re-running train_model.py.

11. Limitations and Ethical Considerations

- **Training data:** the ML model learned from computer-generated fakes. Human-written fakes rely on rule-based heuristics, which indicate suspicion rather than prove it.
- **Language:** the rules, sentiment lexicon and model are English only.
- **Single review:** the detector sees only the text. Platforms use extra signals, such as reviewer history, timing and purchase records, that are not available here.
- **OCR quality:** blurry, cropped or stylized screenshots may be misread; the editable text box mitigates this.
- **Responsible use:** scores are probabilistic and can be wrong. The app states clearly that its verdicts should not be the only basis for decisions about real people or businesses.

12. Future Enhancements

- Train on human-written fake reviews (for example, the Ott et al. deceptive opinion corpus) so the ML model covers both kinds of fake.
- Try a transformer model (such as DistilBERT) and compare it with the current model's accuracy and speed.
- Batch analysis: upload a CSV of reviews and get a summary dashboard.
- Analyze all reviews of a product together to detect coordinated campaigns and near-duplicate text.
- Support more languages.
- Offer a browser extension that checks reviews directly on shopping sites.

13. Conclusion

The Fake Review Detector shows that a simple, fully Python stack can deliver a useful and explainable fake-review checker. Combining a 95%-accurate machine-learning model with eleven transparent rules catches both machine-written and human-written fakes, and the strongest evidence is never diluted. Screenshot support makes the tool practical for everyday shoppers, and the interactive interface makes every verdict easy to understand and question. The project runs at no cost on Streamlit Community Cloud and is covered by automated tests.

References

1. Salminen, J., Kandpal, C., Kamel, A. M., Jung, S., & Jansen, B. J. (2022). Creating and detecting fake reviews of online products. Journal of Retailing and Consumer Services, 64, 102771. Dataset: https://osf.io/tyue9/

2. Ott, M., Choi, Y., Cardie, C., & Hancock, J. T. (2011). Finding Deceptive Opinion Spam by Any Stretch of the Imagination. Proceedings of ACL.

3. Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Proceedings of ICWSM.

4. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12.

5. RapidOCR project: https://github.com/RapidAI/RapidOCR

6. Streamlit documentation: https://docs.streamlit.io

Appendix A. Project Structure

```
fake-review-detector/
  app.py                        Streamlit page
  train_model.py                trains and saves the model
  requirements.txt              pinned dependencies
  detector/
    analyzer.py                 combines the detectors into a verdict
    heuristics.py               eleven rule-based criteria
    model.py                    loads the classifier
    ocr.py                      screenshot to text
  ui/
    components.py               cards, highlights, charts
    style.css                   visual design
  models/review_model.joblib    trained model (2.2 MB)
  samples/                      sample screenshot
  tests/                        automated tests
  .streamlit/config.toml        theme and upload limit
```